УНИВЕРСИТЕТ ИТМО

Домашняя работа №3

Тестирование API средствами Postman

Выполнил:	Камалов Руслан Олегович
Группа:	БР1.2
ИСУ:	408732

Санкт-Петербург

2026

Отчёт по домашней работе №3

Студент: Камалов Руслан Олегович · Группа: БР1.2 · ИСУ: 408732

Цель

Я подготовил комплексный автоматизированный сценарий проверки API в Postman.

Сценарий

Коллекция последовательно проверяет health-check, регистрацию, чтение профиля по JWT, вход администратора и отказ обычному пользователю в административной операции. Затем я создаю тренировку, проверяю фильтр каталога, создаю план, добавляю тренировку, фиксирую выполнение и проверяю агрегаты. После создания и публичного чтения статьи сценарий удаляет тестовые сущности.

Коллекция сохраняет токены и UUID в environment. Уникальные email и slug формируются перед запуском. В каждом запросе есть проверки HTTP-кода и значимых полей ответа.

Коллекция содержит 19 запросов. JSON-файлы collection и environment прошли проверку синтаксиса. Эквивалентный HTTP-сценарий я выполнил на локальном PostgreSQL: регистрация, создание тренировки, плана, записи прогресса и статьи завершились успешно; сводка вернула одну выполненную тренировку и 20 минут.

Вывод

Я реализовал воспроизводимую сквозную проверку позитивных сценариев и RBAC. Файлы находятся в `postman/`.

Приложение

Postman: комплексный сценарий FitFlow

- Запустить локальное приложение и PostgreSQL.
- Убедиться, что существует администратор `admin@fitflow.local` с паролем `AdminPass123`, либо изменить значения в environment.
- Импортировать collection и environment, выбрать `FitFlow Local`.
- Проверить `apiBaseUrl` (`http://localhost:3000/api/v1` при прямом запуске монолита, `http://localhost:8080/api/v1` через Compose) и `healthUrl` (тот же адрес без `/api/v1`). Запустить коллекцию целиком в Collection Runner строго по порядку.

В GitHub Actions обе версии проверяются через Nginx на порту 8080 с временными учебными данными. Для сервера значения URL и пароля администратора передаются отдельно при запуске; настоящий пароль нельзя сохранять в публикуемом environment. Health шлюза проверяет доступность Nginx, остальные запросы проверяют сервисы.

Сценарий генерирует уникальные email и slug, проверяет JWT и RBAC, создаёт тренировку, план, запись прогресса и статью, проверяет агрегаты и публичное чтение, затем удаляет созданные сущности. Для повторного запуска очистить значения `userEmail` и `articleSlug` в environment.